CLAUDE CODE · AGENT SKILLS

wxcc-skills User Guide DRAFT 1

Administer a Webex Contact Center tenant by talking to Claude — read, create, and update teams, users, queues, and more through a library of verified, plain-English skills.

1 Overview

This repo teaches Claude Code how to operate the WxCC Administration REST APIs through a library of *skills* — runbooks Claude loads automatically when your request matches one. You type plain English; Claude picks the skill, runs the verified API recipes inside it, and shows you the results.

> **Draft status.** This guide covers what exists today. Every API call documented in the skills was executed against a live tenant before it was written down — nothing is copy-pasted from docs on faith.

What you can do today

You say	What happens
"How many users do we have? Who's on team X?"	Reads via the Users/Teams APIs
"Which entry point does +1 719 555 0100 route to?"	Dial-number → entry-point lookup
"Create a team called Billing on the Denver site"	Confirms with you, POSTs, verifies, tells you the rollback
"Move agent Jane to the Sales team"	Updates the user's team assignment (site/type rules enforced)
"Add a wrap-up code called Escalated"	Creates the auxiliary code
"What does the Default desktop profile let agents do?"	Reads the Desktop Profile config
"Raise queue X's service level threshold to 30 seconds"	Confirms, PUTs, re-reads to prove it changed

Setup — once per machine, ~15 minutes

Prerequisites: Python 3, Claude Code, a WxCC **administrator** account, and rights to create an app on developer.webex.com.

1. Clone this repo and open it in Claude Code.
2. Create an **Integration** at developer.webex.com with redirect URI `http://localhost:8484/callback` and scopes `cjp:config_read cjp:config cjp:config_write` (read-only? just `cjp:config_read`).
3. Run `cp .env.example .env` and fill in the client ID/secret and your region's API host.
4. Run `python wxcc.py auth login` — a browser opens; sign in as the admin and approve.
5. Check: `python wxcc.py auth status` should show `valid` plus the granted scopes.

Full walkthrough with troubleshooting: the **wxcc-connect** skill.

From then on, just ask Claude for things. Tokens refresh automatically (~14-day access token, ~90-day rolling refresh token).

2 The Safety Model

Reads and writes are treated differently, on purpose:

- **Reads are free.** List/inspect/search operations run without ceremony.
- **Writes always confirm first.** Before any create/update/delete, Claude states exactly what will change and waits for your yes.
- **Every write names its rollback** before it runs (create → delete the new id; update → put the captured original back; delete → flagged as effectively irreversible).
- **Every write is verified by a read.** This API can return 200 while silently ignoring a field — a lesson learned live — so "the API said OK" is never the proof; the re-read is.
- Credentials live in a gitignored `.env`; tokens in a gitignored `.wxcc/`. Nothing secret is ever committed.

3 Skill Catalog

Skill	Covers	Writes?
`wxcc-connect`	OAuth setup, connectivity verification, troubleshooting	—
`wxcc-users` / `-write`	Find/inspect users; update team, site, profiles, CC enablement	UPDATE ONLY
`wxcc-teams` / `-write`	Teams	FULL CRUD
`wxcc-queues` / `-write`	Contact service queues, routing groups, SLTs	FULL CRUD
`wxcc-sites`	Sites	READ-ONLY
`wxcc-entry-points`	Entry points + dial numbers (number↔EP mapping)	READ-ONLY
`wxcc-skill-profiles`	Routing skills + skill profiles	READ-ONLY
`wxcc-aux-codes`	Idle + wrap-up codes	FULL CRUD
`wxcc-address-books`	Address books + entries	FULL CRUD
`wxcc-outdial-ani`	Outbound caller-ID lists	CREATE/DELETE
`wxcc-desktop-profiles`	Desktop Profiles (agent desktop behavior)	UPDATE ONLY

> **Naming note.** Desktop Profiles appear in the API as `agent-profile` — that name is backwards compatibility only. Say "Desktop Profile."

4 How It Works

Every skill drives one shared helper, `wxcc.py` — a dependency-free Python CLI owning OAuth (authorization-code flow as your admin identity), token storage/refresh, org-id resolution, pagination, and authenticated GET/POST/PUT/DELETE against your region's `api.wxcc-*.cisco.com` host. Skills are markdown runbooks with verified commands, field lists, and trap tables (the 404s, silent failures, and validation rules found by probing a live tenant). Longer term, the helper is intended to become an MCP server.

5 Known Limits (honest list)

- User **creation/deletion** is not possible via this API (Control Hub owns identity); names/emails are read-only here.
- Some entities are read-only so far: sites, entry points/dial numbers, skill profiles.
- Flows, desktop layouts, webhooks/subscriptions, bulk import/export, and the GraphQL analytics API are on the roadmap, not built.
- Region host is configured manually (`WXCC_API_BASE`); only us1 has been exercised.
- Anything a skill marks "candidate" has not been run against a live tenant yet.

6 Roadmap

Entry-point/dial-number writes · skill & skill-profile writes · desktop layouts · bulk-export · webhooks · GraphQL analytics (real-time stats) · MCP server packaging · this guide, expanded.

> Draft 1 — 2026-07-11. Feedback welcome: what would you ask it to do first?